YOCO: Cross-Layer KV Sharing

Contents

1. YOCO 1
2. YOCO-U 3
3. YOCO-Sparse 4
4. DeepSeek-V4.1-Flash 5
5. Gemma 4 E2B / E4B 7
6. WeLM 7

With the release of DeepSeek-V4.1-Flash, YOCO has come back into the spotlight. In my view, YOCO is an excellent piece of work, and several interesting variants have emerged from it.

YOCO has two main motivations:

1. **Prefill–Decode (PD) asymmetry.** The main job of Prefill is to build the cache. Only the last prompt token needs to produce the first logits; earlier tokens do not need to compute logits at all. A standard Transformer struggles to fully exploit this asymmetry because each layer needs its own KV Cache. YOCO instead lets the later Cross-Decoder layers share a single global Shared KV, allowing earlier tokens in a long Prefill to exit as soon as the Shared KV has been built. This substantially amplifies the benefits of PD asymmetry. (Su Jianlin has also discussed this on Zhihu.)
2. **Compressing the KV Cache along the layer dimension.** KV Cache size can be viewed approximately as the product of three dimensions: entry (dimensionality and number of heads) × sequence (length) × layer (number of layers). Previous work has already compressed it along different dimensions:
   - **Entry**: GQA/MQA reduce the number of KV heads, while MLA compresses the per-token KV representation dimension d_{KV}.
   - **Sequence**: Attention mechanisms such as CSA/HCA/Linear reduce the caching requirements associated with the history length N.
   - **Layer**: YOCO-style methods target the layer dimension, letting later layers share a single global KV Cache.

1. YOCO

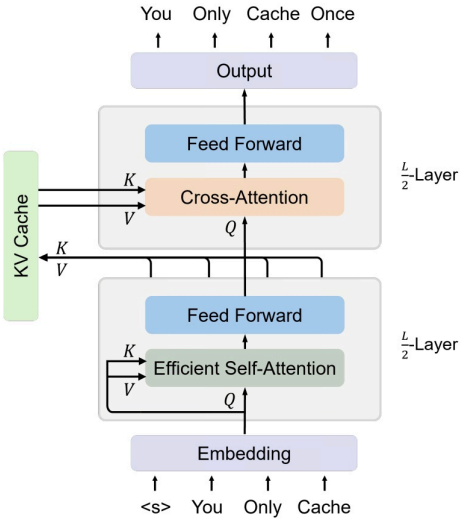


Figure 2: Overview of the decoder-decoder architecture. Self-decoder generates the global KV cache. Then cross-decoder employs cross-attention to reuse the shared KV caches. Both self-decoder and cross-decoder use causal masking. The overall architecture behaves like a decoder-only Transformer, autoregressively generating tokens.

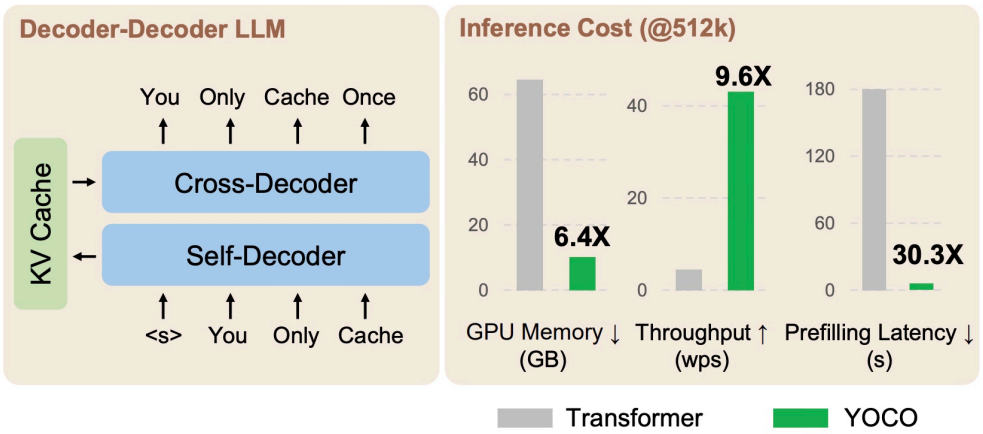


Figure 1: We propose a decoder-decoder architecture, YOCO, for large language model, which only caches key/value once. YOCO markedly reduces the KV cache memory and the prefilling time, while being scalable in terms of training tokens, model size, and context length. The inference cost is reported to be 512K as the context length, and Figures 7–10 present more results for different lengths.

YOCO's architecture is very straightforward: split a standard decoder-only Transformer into two halves.

The first half is the Self-Decoder. More aggressively, this part even uses gated retention or sliding-window attention to further compress the entire context into **a shared global KV Cache**.

The second half is the Cross-Decoder. Each layer no longer stores its own historical K/V. Instead, it uses its own Query to cross-attend to the same shared KV.

The KV Cache cost of a standard Transformer drops from roughly $O(LND)$ to:

$$O((N+L)D)\,.$$

Even better, Prefill does not need to run the later Cross-Decoder at all (which is incredibly useful for the extremely long inputs in today's agent scenarios).

2. YOCO-U

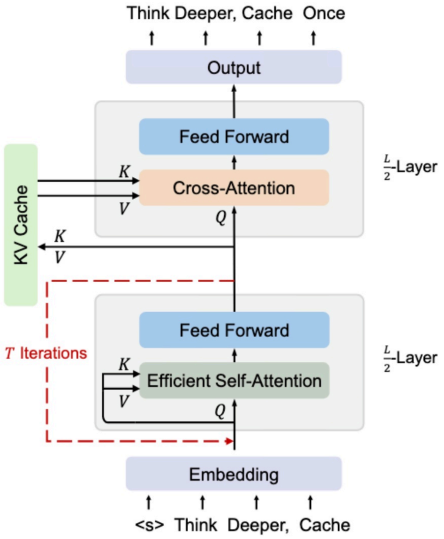


Figure 1: Overview of the YOCO-U architecture. **Universal Self-Decoder** (bottom) performs **recursive computation** (T iterations, indicated by the red dashed line) using efficient self-attention (local windows) to refine representations. Crucially, the global KV cache produced for cross-attention is generated once and remains constant regardless of T, while only the local window-based KV caches grow with iterations, keeping overall cache overhead negligible. **Cross-Decoder** (top) reuses this shared global KV cache via cross-attention for autoregressive token prediction.

YOCO-U adds **Loop** to the earlier Self-Decoder—that idea that has remained shrouded in uncertainty while attracting wave after wave of attention this year. It loops the Self-Decoder three times to produce a Shared KV; the change on the Cross-Decoder side is to use **NOPE**.

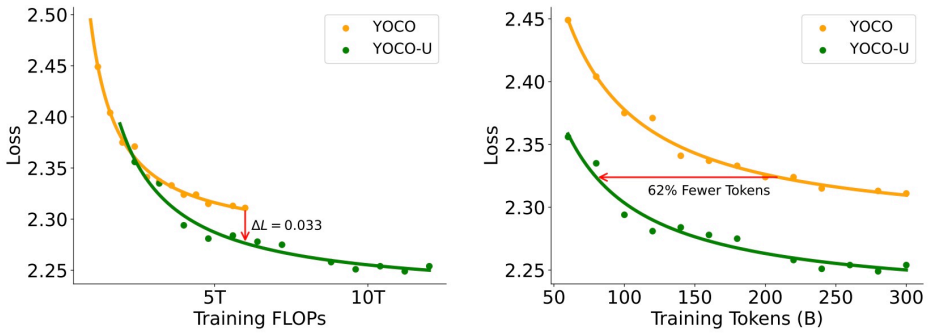


Figure 2: Scaling behavior of language modeling loss with the same model size. **(Left)** Loss versus training FLOPs: YOCO-U achieves competitive or lower loss (ΔL=0.033) at the same FLOPs budget, while incurring negligible KV cache overhead. **(Right)** Loss versus training tokens: YOCO-U also improves data efficiency, requiring approximately 62% fewer tokens to reach comparable performance.

Model	KV Cache Memory	Prefilling Time	Decoding Time
Transformer	$\mathcal{O}(LND)$	$\mathcal{O}(LN^2D)$	$\mathcal{O}(LND)$
YOCO	$\mathcal{O}((N+WL)D)$	$\mathcal{O}(\frac{L}{2}ND)$	$\mathcal{O}(\frac{L}{2}(N+W)D)$
Loop / Universal Transformer	$\mathcal{O}(LTND)$	$\mathcal{O}(LTN^2D)$	$\mathcal{O}(LTND)$
YOCO-U	$\mathcal{O}((N+WTL)D)$	$\mathcal{O}(\frac{L}{2}TND)$	$\mathcal{O}(\frac{L}{2}(N+WT)D)$

Table 1: Inference complexity comparison. N, L, D, T denote sequence length, number of layers, hidden dimension, and loop iterations. W is the local window size in efficient self-attention. In YOCO-U, the global KV cache ($\mathcal{O}(ND)$) is independent of T, and only the local efficient-attention cache ($\mathcal{O}(WTLD)$) scales with iterations. Since $W \ll N$, the additional overhead is negligible for long sequences.

3. YOCO-Sparse

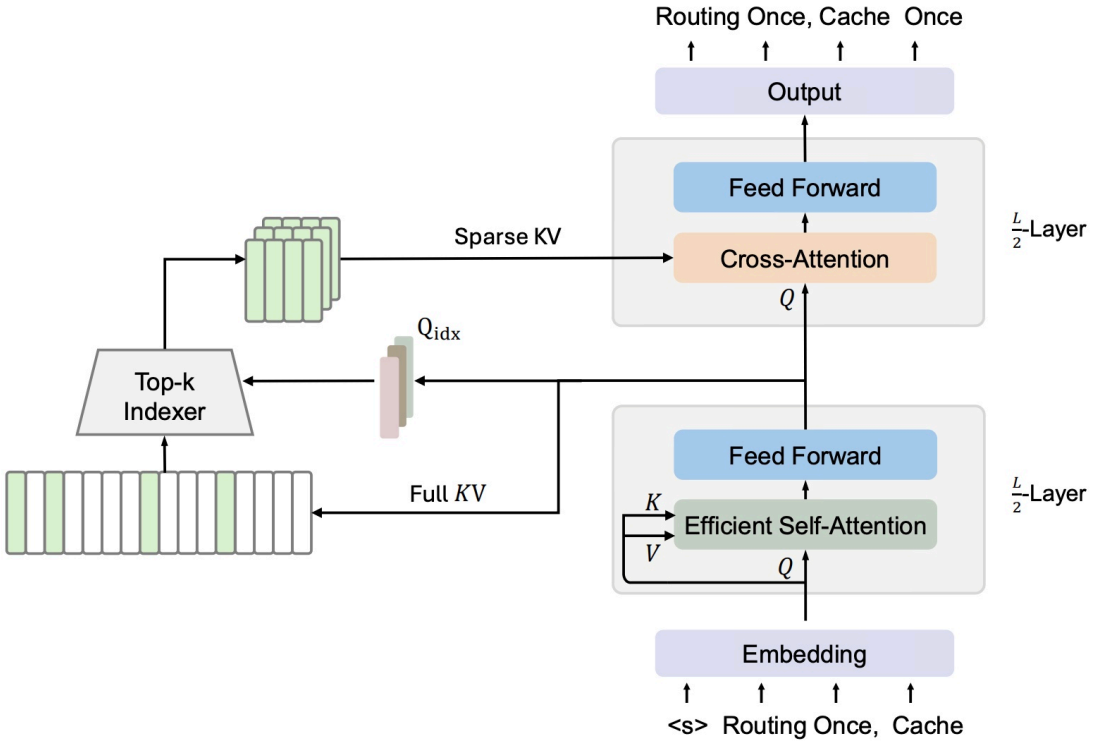


Figure 1: Overview of cross-layer sparse attention. The self-decoder first produces a shared KV cache, which is computed only once and then reused by all subsequent cross-decoder layers. During this stage, a shared query-aware indexer jointly generates the routing queries and keys and computes a token-level sparse top-k index for each query token. This sparse index is also produced only once and is shared across the following cross-decoder layers, allowing them to reuse the same selected KV positions instead of recomputing layer-specific sparse routing.

YOCO-Sparse takes an even more direct approach: since multiple Cross-Decoder layers in YOCO's second half already read the same KV Cache, why not **share the Routing Index for Sparse Attention too**? Savings on top of savings.

Table 1: Comparison of inference complexity. N, L, and D denote sequence length, number of layers, and hidden dimension. W_1 denotes the local window size, and W_2 denotes the number of selected tokens in sparse attention. γ denotes the fraction of global-attention layers in a hybrid model, and η denotes the per query-key routing cost of the indexer.

Model	KV Cache Memory	Prefilling Time	Decoding Time
Transformer	$\mathcal{O}(LND)$	$\mathcal{O}(LN^2D)$	$\mathcal{O}(LND)$
Hybrid TRM	$\mathcal{O}(L(\gamma N + (1-\gamma)W_1)D)$	$\mathcal{O}(L(\gamma N^2 + (1-\gamma)W_1 N)D)$	$\mathcal{O}(L(\gamma N + (1-\gamma)W_1)D)$
YOCO (Dense)	$\mathcal{O}((N + W_1 L)D)$	$\mathcal{O}(\frac{L}{2}W_1 ND)$	$\mathcal{O}(\frac{L}{2}(N + W_1)D)$
DSA	$\mathcal{O}(LND)$	$\mathcal{O}(LW_2 ND + \eta LN^2)$	$\mathcal{O}(LW_2 D + \eta LN)$
YOCO (CLSA)	$\mathcal{O}((N + W_1 L)D)$	$\mathcal{O}(\frac{L}{2}W_1 ND)$	$\mathcal{O}(\frac{L}{2}(W_1 + W_2)D + \eta N)$

Table 2: Main downstream benchmark results for the 4B Transformer baseline, dense YOCO, and YOCO with cross-sparse attention. YOCO (CLSA) maintains the overall capability profile of the dense baselines while improving ARC-Challenge (ARC-C), GSM8K, and DROP, and matching the best HumanEval score.

Model	ARC-C	BBH	GSM8K	HellaSwag	HumanEval	MMLU	DROP	WinoGrande
Transformer	0.453	**0.420**	0.434	0.667	0.384	**0.527**	0.366	**0.638**
YOCO (Dense)	0.461	0.411	0.430	**0.676**	**0.396**	0.519	0.387	0.630
YOCO (CLSA)	**0.465**	0.418	**0.470**	0.674	**0.396**	0.513	**0.391**	0.616

4. DeepSeek-V4.1-Flash

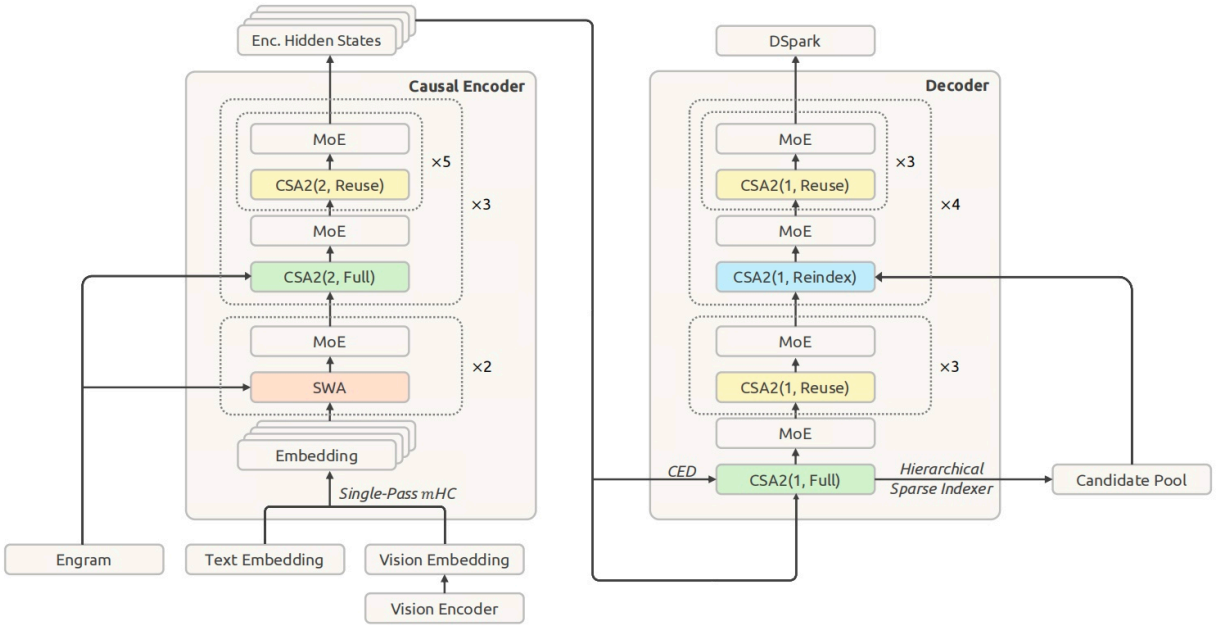


Figure 3 | **Overall architecture of DeepSeek-V4.1-Flash.** The 40-layer network is divided into a causal encoder and a decoder, each with 20 layers. All feed-forward layers use standard DeepSeekMoE. The first two encoder layers use sliding window attention (SWA); the rest use Compressed Sparse Attention 2 (CSA2), with CSA2(ratio, mode) specifying the compression ratio and mode. The model also uses Single-Pass *m*HC, Engram, DSpark, and a Hierarchical Sparse Indexer.

DeepSeek has always been extremely, extremely, extremely aggressive about compressing KV, and this time 4.1 takes it to a truly jaw-dropping level.

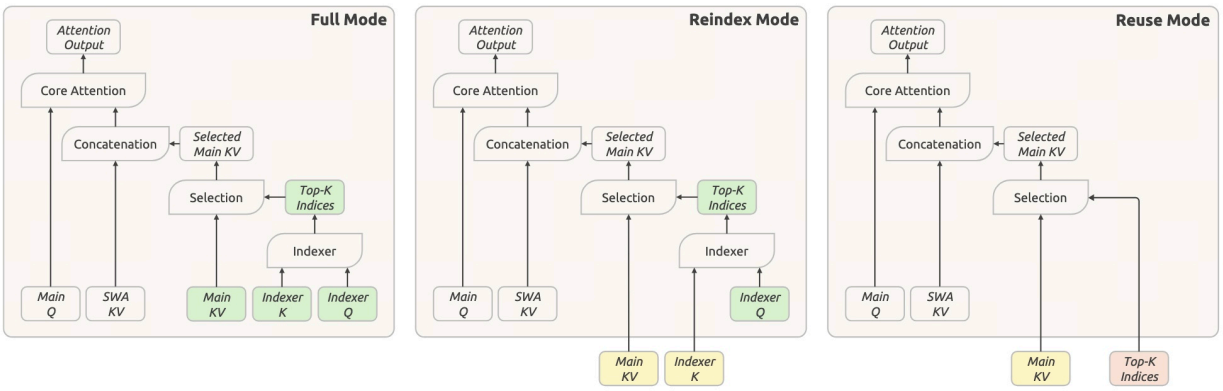


Figure 4 | **Three operating modes of CSA2.** The modes differ in how they obtain main KV, indexer K, and Top-K indices. Green blocks indicate quantities computed in the current layer; yellow blocks indicate main KV and indexer K reused from the most recent Full Mode layer; while red blocks indicate Top-K indices reused from the most recent index-producing (Full or Reindex Mode) layer. All three modes compute main Q and SWA KV in the current layer.

DeepSeek-V4.1-Flash's Attention is roughly **YOCO/CED + CSA2 + SWA**.

YOCO/CED: Reducing Prefill Computation

Global KV follows the overall YOCO framework, splitting the 40 layers into **a 20-layer Causal Encoder + a 20-layer Decoder**. DeepSeek itself calls this the CED architecture.

During Prefill, the first 20 layers run. The Decoder's Global KV is generated directly from the Encoder's final layer, H_{20}, through independent projections for each layer. This avoids running the entire prompt through all of the last 20 layers again.

CSA2: Reusing Global KV and Top-K Across Layers

On top of this, Global Attention uses CSA2 for reuse across layers:

- **Full layers**: Generate new Global KV and Top-K.
- **Reindex layers**: Reuse KV but select Top-K again.
- **Reuse layers**: Reuse both KV and Top-K.

Within the Decoder, only the first Full layer generates Global KV. Subsequent layers share this KV, with Reindex updating the retrieval positions every few layers. Global Sparse Attention ultimately reads only the **Top-512 remote KV entries**.

SWA: Short-Term Caching and Local Reconstruction

To save even more storage, DeepSeek no longer keeps SWA KV in the persistent cache for the long term. What it mainly retains long-term is Global KV, while SWA KV lives only in a host-DRAM pool with a short TTL. **Global Main KV uses FP4, while SWA KV stays in FP8**.

If SWA KV is lost, reconstructing it would normally require replaying $128\times$ layers tokens, since the effective receptive field of stacked SWA layers grows with the number of layers. But DS V4.1 simply **replays only the last 128 tokens** (it is so good at squeezing out savings, I could cry...).

During Decode, each new token still passes through all 40 layers as usual, using **Top-512 Global Sparse KV + 128 Local SWA KV** together.

5. Gemma 4 E2B / E4B

Gemma 4 actually has quite a few architectural tricks, and E2B / E4B also use a technique similar to YOCO.

The idea is simple and direct: **many layers in the later part stop computing their own K/V, keep only their independent Q, and reuse the KV from the last earlier Attention layer of the same type.**

Model	Total Layers	Later Layers Sharing KV	Sliding Attention KV Source	Full / Global Attention KV Source
Gemma 4 E2B	35	L15–L34	KV from L13	KV from L14
Gemma 4 E4B	42	L24–L41	KV from L22	KV from L23

6. WeLM

WeLM also does something similar to YOCO with **KV-Mirror**, but its fundamental goal is not to save KV Cache; it is to **save Prefill computation**.

Specifically, it uses **a U-shaped KV sharing strategy**: the first third of the layers are mirrored to the last third for sharing. What is reused is the mirrored layers' **Hidden States before the K/V projections** (**not the KV!**). The target layers' own projections then recompute K/V, saving the full Attention, MoE, and other computations in the last third of the layers during Prefill.

Why a U shape? An earlier paper, KVSharer, found that **sharing KV between layers that differ more actually leads to better performance**.